

Sistema integral de repositorio y evaluación de trabajos científicos en instituciones educativas

Juan Ignacio Piñeyro Bustamante¹, Abner Gilead Araujo Guedes², Victor Machado Alves³

¹Alumno del curso de Tecnologías en Análisis y Desarrollo de Sistemas. Instituto Federal Sul-río-grandense Campus Sant'ana do Livramento (IFSul)

²Profesor Orientador. Instituto Federal Sul-río-grandense Campus Sant'ana do Livramento (IFSul)

³Profesor Co-orientador. Instituto Federal Sul-río-grandense Campus Sant'ana do Livramento (IFSul)

juanpineyro@gmail.com, abnerguedes@ifsul.edu.br,
victoralves@ifsul.edu.br

Abstract. *This work presents AcademConnect, an automated system for the management and evaluation of scientific papers, developed for educational institutions and research centers. The system allows students to submit their works, professors to act as advisors or co-advisors, and internal or external evaluators to assess projects using reusable evaluation templates. The platform integrates a centralized repository of works with versioning, academic profiles with areas of expertise, and an algorithm that recommends suitable reviewers and advisors by combining thematic affinity, workload balance and availability. By systematizing the evaluation process, the developed system reduces the administrative workload for professors and improves the traceability of academic output, promoting efficiency and transparency in academic management.*

Keywords: *Academic system; Institutional repository; Reviewer assignment algorithms.*

Resumo. *El presente trabajo presenta AcademConnect, un sistema automatizado para la gestión y evaluación de trabajos científicos, desarrollado para instituciones educativas y centros de investigación. El sistema permite a los estudiantes registrar sus trabajos, a los profesores actuar como orientadores o coorientadores, y a evaluadores internos o externos calificar los proyectos mediante plantillas de evaluación reutilizables. La plataforma integra un repositorio centralizado de trabajos con versionamiento, perfiles académicos con áreas de especialización, y un algoritmo que recomienda revisores y orientadores adecuados combinando afinidad temática, equilibrio de carga y disponibilidad. Al sistematizar el proceso de evaluación, el sistema desarrollado reduce la carga administrativa de los docentes y mejora la trazabilidad de la producción académica, promoviendo la eficiencia y la transparencia en la gestión académica.*

Keywords: *Sistema académico; Repositorio institucional; Algoritmos de asignación de revisores.*

1. Introducción

La evaluación de trabajos científicos constituye un pilar fundamental en el ámbito universitario, ya que implica un análisis riguroso de la calidad metodológica, la originalidad y la contribución académica de cada producción. Sin embargo, este proceso suele verse afectado por la falta de herramientas adecuadas que permitan realizar una gestión eficiente y transparente. En muchos casos, la evaluación continúa realizándose de forma manual o mediante sistemas digitales con funcionalidades limitadas, lo que genera retrasos, inconsistencias y una carga operativa innecesaria.

Uno de los principales desafíos es la ausencia de repositorios académicos centralizados, lo cual dificulta el acceso a los trabajos producidos por los estudiantes y limita su visibilidad. La inexistencia de un espacio institucional donde almacenar y exhibir estas producciones afecta tanto la trazabilidad del desarrollo académico como la posibilidad de realizar evaluaciones continuas más precisas y contextualizadas.

Otro problema relevante es la falta de mecanismos eficientes para la asignación de revisores, un desafío ampliamente reconocido en la literatura como el “Problema de Asignación de Revisores” (Aksoy et al., 2023). Esta deficiencia provoca desequilibrios en la carga de trabajo (Stelmakh et al., 2018), inconsistencias en los criterios de evaluación y, en consecuencia, pérdida de eficiencia operativa (Schulz et al., 2022; Kreutz & Schenkel, 2021). La suma de estos factores genera un proceso evaluativo fragmentado y poco estandarizado.

Ante este escenario, la ausencia de un sistema centralizado y dinámico de evaluaciones compromete tanto la transparencia como la equidad en la valoración de los trabajos. Por ello, se vuelve imprescindible desarrollar una solución integral que articule un repositorio institucional, visto como una herramienta clave para la gestión del conocimiento (Barrueco & Termens, 2022), perfiles académicos completos y un flujo de evaluación estructurado. La implementación de un sistema de este tipo no solo moderniza la gestión académica, sino que también contribuye a preservar, organizar y visibilizar la producción científica de la institución.

2. Justificación

La propuesta de desarrollar un sistema integral de gestión académica se justifica por su potencial para mejorar significativamente la calidad y eficiencia de los procesos institucionales relacionados con la evaluación de trabajos científicos (Schulz et al., 2022). Al integrar un repositorio institucional, perfiles académicos y un módulo estructurado de evaluaciones, la institución obtiene una herramienta capaz de estandarizar procedimientos, reducir la carga administrativa y fortalecer la toma de decisiones basada en datos (Lynch, 2003).

El sistema contribuye directamente a la consolidación del conocimiento institucional, permitiendo que las producciones científicas de los estudiantes permanezcan accesibles y organizadas (Crow, 2002). Esto no solo preserva la memoria académica, sino que también facilita la investigación, la supervisión de trayectorias formativas y la visibilidad del trabajo estudiantil. Estas funciones coinciden con los beneficios tradicionalmente atribuidos a los repositorios institucionales, los cuales mejoran la preservación digital, la difusión del conocimiento y la accesibilidad académica (Lynch, 2003).

Asimismo, la automatización de la asignación de revisores y la centralización de criterios evaluativos generan un proceso más ágil, transparente y coherente. Estas mejoras promueven un ambiente académico más equitativo, aumentan la confiabilidad del proceso de evaluación y reducen los posibles sesgos derivados de la gestión manual (Aksoy et al., 2023). Investigaciones en sistemas de gestión de conferencias y evaluación por pares demuestran que los algoritmos de asignación automática de revisores incrementan la equidad y mejoran la eficiencia operativa al considerar afinidad temática y distribución equilibrada de la carga de revisión (Stelmakh et al., 2018). La evidencia empírica respalda que estos sistemas optimizan la asignación considerando tanto la competencia temática como la disponibilidad de los revisores (Kreutz & Schenkel, 2021).

En conjunto, el proyecto presenta una solución pertinente y alineada con las demandas contemporáneas de modernización institucional, aportando beneficios tangibles tanto para docentes como para estudiantes, y contribuyendo a elevar la calidad del proceso académico como un todo. Este tipo de sistemas se considera fundamental para instituciones que buscan fortalecer procesos de evaluación, preservación del conocimiento y transparencia operativa dentro de entornos educativos altamente digitalizados (Crow, 2002).

3. Objetivos

3.1. Objetivo General

Desarrollar un sistema integral de gestión académica que optimice y automatice los procesos de evaluación de trabajos científicos, integrando un repositorio institucional, un módulo de evaluaciones dinámicas y perfiles académicos completos para mejorar la eficiencia, transparencia y calidad de la valoración académica.

3.2. Objetivos Específicos

1. Crear un repositorio académico centralizado que permita almacenar, organizar y gestionar las producciones científicas institucionales;
2. Desarrollar un sistema de evaluaciones dinámico que permita definir y gestionar criterios personalizados según el tipo de trabajo académico;
3. Generar métricas del proceso evaluativo que apoyen la toma de decisiones institucionales.

4. Fundamentación Teórica

La gestión, evaluación y preservación de la producción científica en instituciones de educación superior ha adquirido una relevancia creciente en los últimos años, impulsada por la expansión del acceso abierto, la digitalización y la necesidad de garantizar procesos evaluativos más eficientes y transparentes. En este contexto, tres ejes conceptuales sustentan el desarrollo del sistema propuesto: los repositorios institucionales, la automatización del proceso de evaluación académica y los algoritmos para la asignación de revisores.

4.1. Repositorios institucionales y preservación del conocimiento

Los repositorios institucionales se han consolidado como herramientas fundamentales para la gestión del conocimiento académico (Crow, 2002). Su función principal es

almacenar, preservar y difundir la producción intelectual de una institución, garantizando su accesibilidad a largo plazo. Los repositorios institucionales fortalecen la memoria científica institucional y facilitan la comunicación académica, permitiendo una mayor visibilidad y disponibilidad de los trabajos universitarios.

Asimismo, Barrueco y Termens (2022) señalan que estos repositorios no solo incrementan la visibilidad externa de la institución, sino que también optimizan la preservación digital mediante políticas específicas y el uso de estándares internacionales.

En conjunto, la literatura sostiene que los repositorios institucionales no solo cumplen una función de archivo, sino que aportan a la evaluación longitudinal del desempeño académico de estudiantes y docentes, un aspecto clave en sistemas integrados de gestión evaluativa (Lynch, 2003).

4.2. Automatización en la evaluación de trabajos científicos

El proceso tradicional de revisión académica suele depender de decisiones manuales al asignar revisores, lo que puede generar una sobrecarga para ciertos expertos, desequilibrios en la carga de trabajo y falta de optimización. Por ello, existe una creciente investigación en algoritmos automatizados que tomen en cuenta no solo la afinidad temática entre el manuscrito y el revisor, sino también otros criterios operativos como la disponibilidad o la carga de trabajo.

En particular, el problema conocido como *Reviewer Assignment Problem* (RAP) ha sido ampliamente estudiado. Aksoy, Yanik y Amasyali (2023) realizan una revisión sistemática de este problema, identificando técnicas que optimizan la asignación de revisores considerando tanto la competencia temática como restricciones operativas.

Entre las soluciones destacadas se encuentra *PeerReview4All*, un algoritmo diseñado para optimizar simultáneamente la afinidad temática y la equidad en la distribución de revisores. Según Stelmakh, Shah y Singh (2018), este método garantiza que incluso los trabajos con menor afinidad inicial reciban evaluadores adecuados, manteniendo altos niveles de precisión.

Otra propuesta importante es *RevASIDE*, un sistema de recomendación para asignar un conjunto de revisores que no solo se ajusten en experiencia, sino que también aporten diversidad temática y autoridad, lo que permite conformar equipos equilibrados (Kreutz & Schenkel, 2021).

Además, la evaluación de la similitud entre un revisor y un manuscrito se mejora con *datasets* de referencia, lo que evidencia cómo incluso los algoritmos actuales tienen margen para mejorar en la predicción de la experiencia real y la idoneidad del revisor.

En suma, la evidencia teórica y empírica respalda la creación de un algoritmo que combine afinidad temática, autoridad académica y disponibilidad del revisor para lograr asignaciones más justas, eficientes y equilibradas.

4.3. Síntesis

La literatura reciente evidencia la necesidad de sistemas integrales que combinen rubricas personalizadas de evaluación, repositorios institucionales sólidos (Crow, 2002; Lynch, 2003) y algoritmos avanzados para la asignación de revisores (Aksoy et al., 2023).

Estos elementos contribuyen de manera significativa a la eficiencia, transparencia y trazabilidad del proceso de evaluación académica, alineándose con las tendencias actuales de modernización de la gestión universitaria.

5. Aporte teórico

El problema de asignación de revisores (*Reviewer Assignment Problem*, RAP) se ha estudiado sobre todo para conferencias y revistas, donde algoritmos como *PeerReview4All* (Stelmakh et al., 2018) y *RevASIDE* (Kreutz & Schenkel, 2021) asignan muchos manuscritos a la vez optimizando la afinidad temática y el equilibrio de la carga. Estas soluciones priorizan el resultado global de la asignación y ofrecen poca explicación sobre por qué se elige a cada revisor.

La evaluación de trabajos de conclusión de curso y tesis en una institución presenta condiciones diferentes. El conjunto de evaluadores es pequeño y combina docentes internos con evaluadores externos, la comunidad académica necesita poder justificar cada asignación y existen restricciones de integridad, como evitar que el orientador de un trabajo participe en su evaluación. Por ello, en AcademConnect la idoneidad de un revisor se modela como una combinación de factores explícita y configurable, en la que cada sugerencia puede justificarse a partir de los valores que la producen.

5.1. Modelo conceptual de idoneidad del revisor

A partir de esa postura, y en línea con los criterios de afinidad y carga estudiados en la literatura sobre asignación de revisores (Aksoy et al., 2023; Stelmakh et al., 2018), se propone un modelo conceptual que entiende la idoneidad de un evaluador para un trabajo como la composición de tres dimensiones complementarias:

- Afinidad temática: grado de correspondencia entre las áreas de conocimiento del trabajo y la experticia declarada del evaluador.
- Equidad de carga: preferencia por los evaluadores menos ocupados, de modo que la asignación distribuya el esfuerzo de revisión y evite la sobrecarga de unos pocos expertos.
- Disponibilidad: capacidad efectiva del evaluador para asumir una nueva revisión.

El modelo postula que estas dimensiones no tienen un peso universal, sino que su importancia relativa debe ser un parámetro definible por la institución según su política de evaluación. Esta parametrización constituye el núcleo de la perspectiva propia: traslada la decisión sobre qué privilegiar —pertinencia, equilibrio o disponibilidad— desde el algoritmo hacia el criterio institucional.

5.2. Adaptación del RAP al contexto institucional

El modelo introduce dos elementos que las formulaciones orientadas a conferencias no contemplan. El primero son las restricciones de integridad académica: al armar la lista de candidatos, AcademConnect descarta al orientador del propio trabajo y a cualquier evaluador con un conflicto de interés registrado. El segundo es la trazabilidad de las sugerencias, ya que cada recomendación guarda los valores de afinidad, carga y disponibilidad que la generaron y permite explicar por qué se propuso a un evaluador. Esta posibilidad de justificar las asignaciones atiende una limitación reconocida de la revisión manual, donde los criterios suelen quedar implícitos (Aksoy et al., 2023).

El aporte del trabajo, por lo tanto, está en cómo se plantea la idoneidad del revisor: una combinación de afinidad, carga y disponibilidad cuyos pesos configura la institución y cuyas sugerencias pueden trazarse, aplicada a la evaluación de trabajos institucionales y a sus restricciones de integridad. Su formalización matemática y su implementación se presentan en la sección de Resultados.

5.3. Trabajos relacionados

Con el objetivo de comprender el estado del arte y las limitaciones de las soluciones existentes, se realizó un estudio comparativo de plataformas ampliamente utilizadas en contextos académicos (Lynch, 2003). Este análisis permitió identificar fortalezas, limitaciones y patrones funcionales en los sistemas existentes, orientando el diseño de *AcademConnect* hacia una solución más completa y adecuada a las necesidades institucionales.

Open Journal Systems (OJS) es una plataforma de código abierto desarrollada por el Public Knowledge Project, diseñada específicamente para la gestión editorial de revistas académicas (Willinsky, 2005). El sistema facilita el proceso de revisión por pares mediante un flujo de trabajo estructurado que incluye envío de manuscritos, asignación de revisores y comunicación entre editores, autores y evaluadores. OJS ofrece un repositorio editorial estructurado que permite almacenar y organizar artículos publicados, con soporte para metadatos y estándares de publicación académica (Lynch, 2003). Sin embargo, su enfoque está principalmente orientado a la gestión de revistas científicas, lo que limita su aplicabilidad para otros tipos de trabajos académicos institucionales como tesis, proyectos de grado o trabajos de investigación estudiantil. Además, la asignación automática de revisores requiere la instalación de *plugins* adicionales y, por defecto, el proceso es manual, lo que puede generar sobrecarga administrativa en instituciones con alto volumen de trabajos (Aksoy et al., 2023).

EasyChair es una plataforma web ampliamente utilizada para la gestión de conferencias académicas y procesos de revisión de artículos científicos. El sistema se destaca por su capacidad de asignación automática de revisores basada en afinidad temática y disponibilidad, lo que optimiza la distribución de la carga de trabajo entre evaluadores (Kalmukov, 2011). EasyChair ofrece perfiles académicos completos que permiten a los usuarios registrar sus áreas de especialización, historial de publicaciones y preferencias de revisión, facilitando así la identificación de revisores adecuados (Stelmakh et al., 2018). No obstante, su diseño está orientado principalmente a conferencias y eventos académicos temporales, no a repositorios institucionales permanentes (Crow, 2002). El sistema almacena los envíos durante el proceso de revisión, pero no proporciona un repositorio institucional estructurado que preserve y organice la producción académica a largo plazo, limitando su utilidad para la gestión continua de trabajos estudiantiles.

Moodle es una plataforma de aprendizaje virtual (*e-learning*) de código abierto ampliamente adoptada en instituciones educativas (Cole & Foster, 2007). El sistema ofrece una gestión integral de cursos, con funcionalidades para administrar estudiantes, docentes, materiales didácticos y actividades de evaluación pedagógica. Moodle incluye perfiles académicos para estudiantes y docentes, permitiendo el seguimiento de trayectorias formativas y la gestión de calificaciones (Lynch, 2003). Sin embargo, su enfoque está centrado en la evaluación pedagógica y el aprendizaje en línea, no en la evaluación

científica de trabajos de investigación. El sistema permite almacenar archivos por curso, pero no proporciona un repositorio institucional estructurado para trabajos científicos ni mecanismos de revisión por pares académica, lo que limita su aplicabilidad para la gestión de producción científica institucional (Crow, 2002).

El Cuadro 1 presenta una comparación funcional entre *Open Journal Systems (OJS)*, *EasyChair*, *Moodle* y *AcademConnect*, permitiendo visualizar de manera clara las diferencias y limitaciones de cada plataforma, así como destacar las mejoras que aporta la solución propuesta. Esta comparación analiza cinco dimensiones funcionales clave: evaluación de trabajos científicos, repositorio de trabajos, perfiles académicos, asignación automática de revisores y gestión de usuarios y roles. El análisis revela que mientras *OJS* se enfoca principalmente en la gestión editorial de revistas, *EasyChair* está orientado a conferencias temporales, y *Moodle* se centra en la evaluación pedagógica, ninguna de estas plataformas ofrece una solución integral que combine repositorio institucional permanente, evaluación científica con criterios dinámicos y asignación automática de revisores en un único sistema (Schulz et al., 2022). *AcademConnect* se posiciona como una alternativa que integra todas estas funcionalidades, proporcionando una solución completa para la gestión académica institucional. Esta comparación se basa en el análisis de documentación oficial de cada plataforma y estudios comparativos sobre sistemas de gestión académica (Willinsky, 2005; Kalmukov, 2011; Cole & Foster, 2007).

Cuadro 1. Comparación funcional entre plataformas académicas

Funcionalidad	OJS	EasyChair	Moodle	AcademConnect
Evaluación de trabajos científicos	Parcial: evaluación por pares solo para artículos de revista	Parcial: revisión por pares limitada a trabajos de conferencias	No: solo evaluación pedagógica, no científica	Sí: evaluación integral con criterios dinámicos para trabajos institucionales
Repositorio de trabajos científicos	Sí: repositorio editorial estructurado	Parcial: almacenamiento de envíos sin repositorio institucional	No: solo archivos por curso	Sí: repositorio institucional completo con metadatos
Perfil académico	Parcial: perfiles básicos	Sí: perfiles académicos completos para conferencias	Sí: perfiles académicos de estudiantes y docentes	Sí: perfiles extendidos con áreas de especialización
Asignación automática de revisores	Parcial: requiere plugins, manual por defecto	Sí: asignación automática por afinidad y disponibilidad	No: no posee revisión científica	Sí: algoritmo por afinidad temática y carga docente
Gestión de usuarios y roles	Sí: roles editoriales	Sí: roles de conferencia	Sí: roles educativos	Sí: roles ampliados y configurables

6. Metodología

6.1. Metodología de la investigación

6.1.1. Enfoque metodológico

El proyecto se desarrolló bajo un enfoque de Investigación, Desarrollo e Innovación (I+D+I), combinando revisión bibliográfica, análisis comparativo y construcción de una solución tecnológica aplicada. Este enfoque permitió comprender el estado del arte de los sistemas de evaluación académica y diseñar un sistema integral acorde a las necesidades institucionales.

6.1.2. Revisión bibliográfica

La primera fase consistió en una revisión sistemática de literatura orientada a tres ejes principales: (1) procesos de evaluación académica, (2) sistemas de gestión y repositorios institucionales y (3) algoritmos para la asignación de revisores. La búsqueda de literatura específica del dominio —evaluación por pares automatizada y asignación de revisores— se acotó al período 2018-2024, con el objetivo de capturar los desarrollos más recientes. Las obras fundacionales sobre repositorios institucionales y métricas de similitud, así como la documentación técnica y normativa de las tecnologías empleadas, se citan como base conceptual y de implementación por su carácter de referencia establecida, con independencia de esa ventana temporal. Del mismo modo, las referencias canónicas de las plataformas analizadas en los trabajos relacionados —OJS, Moodle y EasyChair— se citan como documentación de sistemas establecidos, con independencia de su año de publicación.

Se consultaron bases de datos como *IEEE Xplore*, *ACM Digital Library*, *Scopus*, *SpringerLink* y *arXiv*, identificando estudios relevantes como Kreutz y Schenkel (2021), cuyos trabajos analizan modelos de recomendación de revisores basados en afinidad temática y disponibilidad. Esta revisión permitió identificar brechas frecuentes en sistemas existentes, tales como la falta de repositorios integrados, criterios de evaluación personalizables y módulos robustos de asignación automática.

6.2. Metodología de desarrollo

6.2.1. Levantamiento de requisitos

El levantamiento de requisitos se realizó a partir de la revisión bibliográfica, el análisis funcional de sistemas similares e información derivada de procesos internos de evaluación en instituciones educativas. Los requisitos se clasificaron en:

- Requisitos funcionales: gestión de trabajos, configuración de criterios, perfiles académicos, asignación automática de revisores, administración del repositorio y generación de informes.
- Requisitos no funcionales: seguridad, rendimiento, disponibilidad, usabilidad, mantenibilidad y escalabilidad.

Este conjunto de requisitos sirvió como base para el modelado conceptual y para la planificación del desarrollo.

6.2.2. Diseño de la arquitectura

El desarrollo de *AcademConnect* se estructuró siguiendo una arquitectura en capas (*three-tier architecture*), con separación clara entre la capa de presentación (*frontend*), la lógica de negocio (*backend*) y la capa de persistencia de datos. En el *backend*, esta organización se materializa en una estructura *controller* → *service* → *repository*, que aísla la exposición de la *API* (Interfaz de Programación de Aplicaciones), la lógica de negocio y el acceso a datos. Esta separación de responsabilidades facilita el mantenimiento, la testabilidad y la reutilización de componentes, y permite que cada capa evolucione de forma independiente (Fowler, 2002).

6.2.3. Selección de tecnologías

Para el desarrollo del sistema se seleccionaron Java como lenguaje del *backend*, Spring Boot como *framework* de servicios, Angular para el *frontend* y PostgreSQL como sistema de gestión de bases de datos, junto con un conjunto de herramientas de soporte para seguridad, persistencia y pruebas. El criterio de selección priorizó la adecuación a los requisitos no funcionales del proyecto —seguridad, escalabilidad y mantenibilidad— y la madurez del ecosistema de cada tecnología. La justificación detallada de cada elección se presenta en la sección Herramientas y métodos.

6.2.4. Diseño y modelo de desarrollo

Para el desarrollo se adoptó un modelo incremental e iterativo, estructurado en ciclos que incluyeron planificación, diseño, implementación, pruebas y revisión. Este enfoque permitió integrar funcionalidades progresivamente y realizar mejoras continuas sobre la base de la validación técnica y del análisis comparativo.

6.2.5. Pruebas y validación

El sistema fue sometido a distintos niveles de prueba:

- Pruebas unitarias: verificación individual de cada componente del sistema, incluyendo métodos, funciones y clases. Estas pruebas se realizaron utilizando *frameworks* de testing como JUnit para el *backend* Java y Vitest para el *frontend* Angular, asegurando que cada unidad de código funcione correctamente de forma aislada antes de la integración.
- Pruebas funcionales: verificación del cumplimiento de los requisitos funcionales.
- Pruebas de integración: validación de la comunicación entre módulos como repositorio, perfiles, evaluaciones y recomendación de revisores.
- Pruebas de aceptación: revisión del flujo general de operación del sistema.
- Validación conceptual: comparación del sistema con los hallazgos de la revisión bibliográfica y del análisis de plataformas similares.

7. Herramientas y métodos

7.1. Java

El *backend* se desarrolló en Java 25. La elección se apoya en la madurez de su ecosistema: sobre Java se construyen Spring Boot y las bibliotecas de persistencia (*JPA*) y de seguridad que emplea AcademConnect, lo que reduce el esfuerzo de integración entre componentes. Además, la máquina virtual de Java (*JVM*) ofrece portabilidad entre entornos y una gestión de memoria acorde a la carga de un sistema de gestión académica (Gosling et al., 2025; Lindholm et al., 2025).

7.2. Spring Boot

Spring Boot es un *framework* derivado del ecosistema Spring que permite crear aplicaciones Java de forma simplificada mediante configuración automática, inyección de dependencias y una arquitectura modular (Walls, 2015). Su enfoque *convention over configuration* reduce el tiempo de desarrollo y facilita la implementación de *APIs REST* (Interfaz de Programación de Aplicaciones de Transferencia de Estado Representacional). Spring Boot elimina la necesidad de configuración XML compleja mediante el uso de anotaciones y configuración basada en Java, lo que acelera significativamente el proceso de desarrollo de aplicaciones empresariales (Gutierrez, 2019).

La elección de Spring Boot se fundamenta en su capacidad para estructurar el *backend* según principios modernos de ingeniería de software, brindando soporte nativo para seguridad, pruebas automatizadas, persistencia con *JPA* (Java Persistence API) *Hibernate* y gestión de controladores *REST*. Estas características lo hacen especialmente adecuado para construir sistemas académicos que requieren organización interna clara, escalabilidad y mantenimiento prolongado (Gutierrez, 2019). En *AcademConnect* se empleó Spring Boot 4, que mantiene este enfoque sobre versiones recientes de Java.

7.3. Angular

Angular es un *framework* para aplicaciones web basado en *TypeScript* que permite construir interfaces dinámicas, modulares y mantenibles. Su arquitectura basada en componentes, el uso de servicios y la tipificación estricta favorecen la creación de sistemas robustos con énfasis en la organización del código. Angular implementa un sistema de inyección de dependencias que facilita la gestión del estado de la aplicación y la comunicación entre componentes, mejorando la mantenibilidad y testabilidad del código (Fain & Moiseev, 2018).

Para este proyecto, Angular se selecciona por su capacidad para manejar interfaces complejas, tales como formularios dinámicos de evaluación, paneles administrativos, perfiles académicos y visualización de métricas. Además, su estructura facilita la comunicación con *APIs REST*, lo cual permite una integración eficiente con el *backend* desarrollado en Spring Boot. El *framework* Angular proporciona herramientas integradas para el desarrollo de formularios reactivos, validación de datos y manejo de eventos, características esenciales para sistemas de gestión académica que requieren interacción constante con el usuario (Fain & Moiseev, 2018). El proyecto utilizó Angular 21, con componentes *standalone* y tipado estricto en *TypeScript*.

7.4. PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto reconocido por su estabilidad, conformidad con estándares y soporte avanzado para tipos de datos complejos. Su motor permite diseñar esquemas robustos, relacionar entidades y garantizar integridad referencial, aspectos fundamentales para sistemas académicos (Obe & Hsu, 2017). PostgreSQL cumple con el estándar SQL y ofrece características avanzadas como transacciones *ACID*, vistas materializadas, funciones almacenadas y disparadores (*triggers*), lo que lo convierte en una solución robusta para aplicaciones que requieren integridad de datos y consistencia transaccional (PostgreSQL, 2023).

Se selecciona PostgreSQL debido a su rendimiento en aplicaciones con alta lectura y escritura de datos, como repositorios de trabajos científicos, registros de usuarios, historial de evaluaciones y metadatos (PostgreSQL, 2023). Además, incorpora funcionalidades como búsquedas *full-text*, índices personalizados y un amplio ecosistema de extensiones, lo que permite desarrollar módulos avanzados y escalables en el futuro (Obe & Hsu, 2017). En el proyecto se utilizó PostgreSQL 16.

7.5. Seguridad y herramientas de soporte

Además de las tecnologías centrales, el desarrollo de *AcademConnect* incorporó un conjunto de herramientas que sostienen requisitos no funcionales clave. La seguridad se implementó con *Spring Security*, mediante autenticación basada en *JSON Web Tokens* (JWT) firmados con *HMAC-SHA256* (Jones et al., 2015) y un servidor de recursos conforme al marco de autorización *OAuth 2.0* (Hardt, 2012), lo que permite controlar el acceso según los roles del sistema (estudiante, docente, evaluador y administrador).

La integridad de la base de datos se gestionó con *Flyway*, que versiona y aplica las migraciones de esquema de forma reproducible, garantizando la consistencia entre entornos. El mapeo entre entidades de dominio y objetos de transferencia de datos (*DTO*) se resolvió con *MapStruct* y *Lombok*, reduciendo el código repetitivo y los errores de conversión manual. Las pruebas de integración se apoyaron en *Testcontainers*, que levanta una instancia real de PostgreSQL en un contenedor efímero durante la ejecución de los *tests*, evitando las discrepancias propias de usar una base de datos en memoria. El empaquetado y despliegue se estandarizaron con *Docker* y *Docker Compose* (Merkel, 2014), que encapsulan la aplicación y sus dependencias en contenedores reproducibles. Finalmente, el manejo de errores de la *API* sigue el formato estandarizado *Problem Details for HTTP APIs* (Nottingham et al., 2023), que devuelve respuestas de error estructuradas y uniformes.

8. Resultados

El desarrollo de *AcademConnect* dio como resultado un sistema funcional que implementa los componentes definidos en los objetivos: un repositorio institucional de trabajos con versionamiento, un módulo de evaluaciones con plantillas de criterios configurables, perfiles académicos con áreas de especialización y el algoritmo de recomendación de revisores y orientadores. Esta sección presenta la formalización del modelo de recomendación, el modelado del sistema y el funcionamiento de los módulos construidos.

8.1. Formalización del modelo de recomendación

El modelo conceptual de idoneidad descrito en el Aporte teórico se formalizó como una función de puntuación. Para un trabajo dado, a cada candidato c (docente o evaluador externo activo) se le asigna:

$$score(c) = w_1 \cdot af(c) + w_2 \cdot (1 - cn(c)) + w_3 \cdot disp(c) \quad (1)$$

donde $af(c)$, $cn(c)$ y $disp(c)$ son la afinidad temática, la carga normalizada y la disponibilidad. Los pesos w_1, w_2, w_3 son parámetros configurables (por defecto 0,6, 0,3 y 0,1). La afinidad se calcula con el índice de Jaccard (Jaccard, 1912) entre las áreas del trabajo (A_t) y las del candidato (A_c):

$$af(c) = \frac{|A_t \cap A_c|}{|A_t \cup A_c|} \quad (2)$$

La carga normalizada $cn(c) = carga(c) / \max_j carga(j)$ expresa la ocupación relativa respecto del candidato más cargado, siendo $carga(c)$ el número de asignaciones activas; el término $(1 - cn(c))$ favorece a los menos ocupados. En la versión actual la disponibilidad se fija en 1,0 y queda como punto de extensión para futuras versiones. El sistema ordena a los candidatos por su puntuación, selecciona los k primeros y guarda, para cada uno, los valores de afinidad, carga y disponibilidad utilizados, de modo que más tarde puede reconstruirse cómo se llegó a cada sugerencia.

La recomendación de orientadores aplica el mismo principio con una función reducida $score_o(p) = w_{o1} \cdot af(p) + w_{o2} \cdot (1 - cn(p))$ (pesos por defecto 0,7 y 0,3), donde la carga es la cantidad de trabajos no finalizados que el docente orienta.

8.2. Modelado del sistema

El modelado del sistema se llevó a cabo utilizando principios de ingeniería de software, permitiendo representar de manera estructurada la arquitectura y el comportamiento del sistema antes de su implementación (Fowler, 2002). Se elaboraron diagramas que facilitan la comprensión de los componentes principales, sus relaciones y las interacciones entre los diferentes actores del sistema (Booch et al., 2005).

El Diagrama de Clases UML (Lenguaje Unificado de Modelado) modela la estructura estática del sistema, mostrando las principales clases, sus atributos y las relaciones entre ellas (Booch et al., 2005). Dada la cantidad de entidades del dominio, el modelo se presenta en tres vistas complementarias que facilitan su lectura.

La Figura 1 presenta la vista de usuarios y perfiles: la jerarquía de *Usuario* —especializada en estudiantes, profesores, evaluadores externos y administradores— y su asociación con las áreas temáticas que definen la experticia de cada usuario.

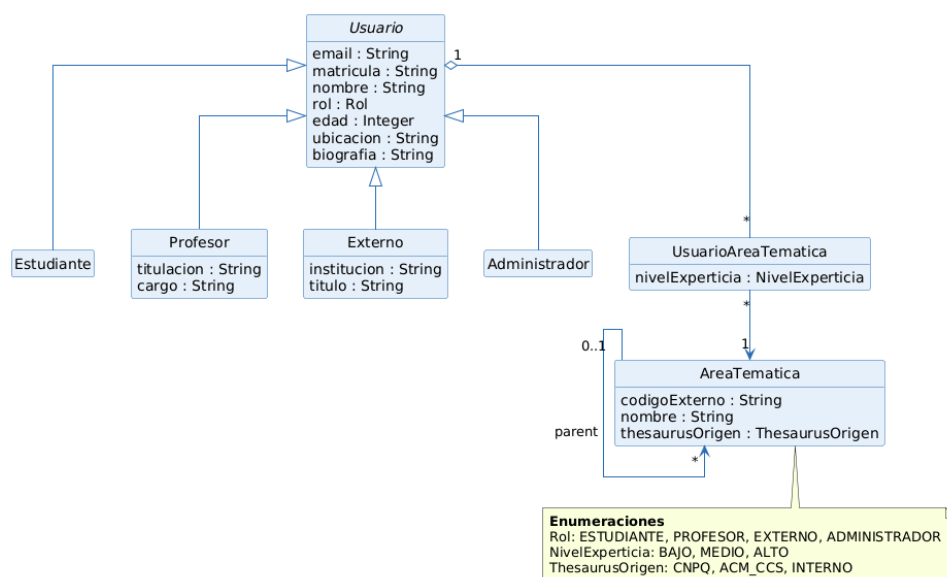


Figura 1. Diagrama de clases — Usuarios y perfiles, Fuente: elaboración propia

La Figura 2 muestra la vista de trabajos, repositorio y orientación: el *Trabajo* como entidad central, su versionamiento y los documentos asociados, junto con los mecanismos de orientación (coorientadores, solicitudes de vinculación e invitaciones).

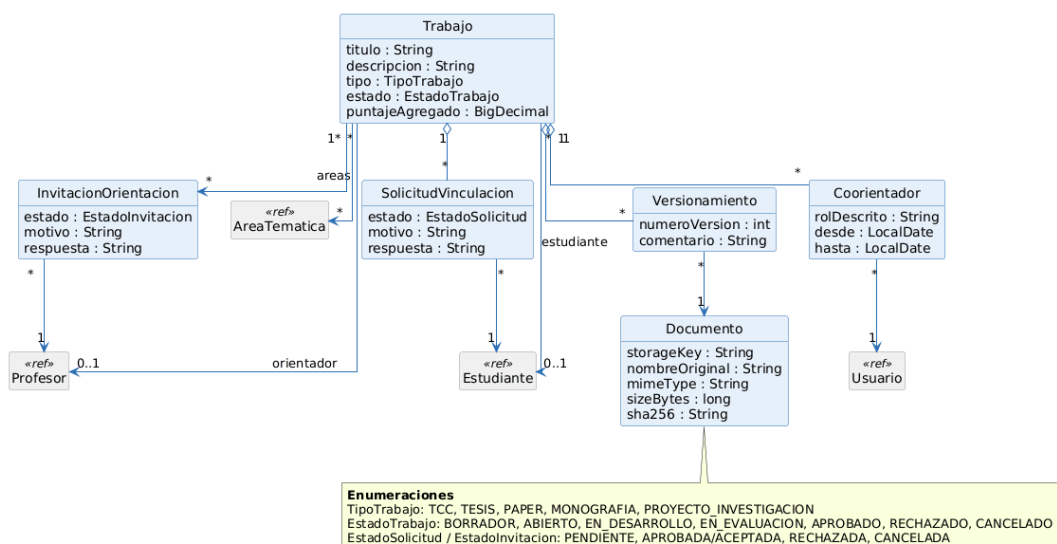


Figura 2. Diagrama de clases — Trabajos, repositorio y orientación, Fuente: elaboración propia

La Figura 3 detalla el flujo de evaluación: las plantillas de evaluación, las asignaciones de evaluadores, las evaluaciones con sus calificaciones por criterio y las sesiones de evaluación.

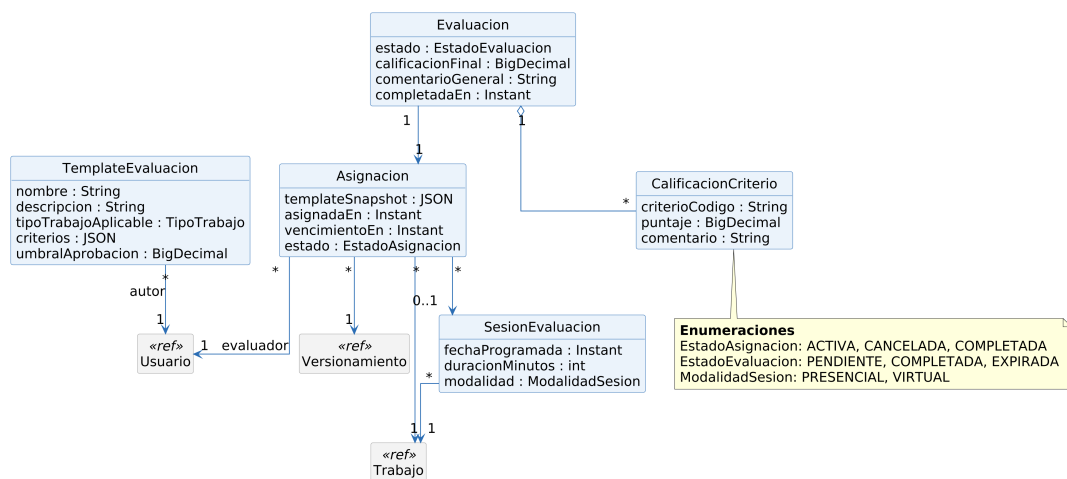


Figura 3. Diagrama de clases — Flujo de evaluación, Fuente: elaboración propia

La Figura 4 presenta las entidades que sustentan el algoritmo de recomendación: las recomendaciones de evaluadores con su puntuación y factores, los conflictos de interés y la disponibilidad declarada por cada evaluador.

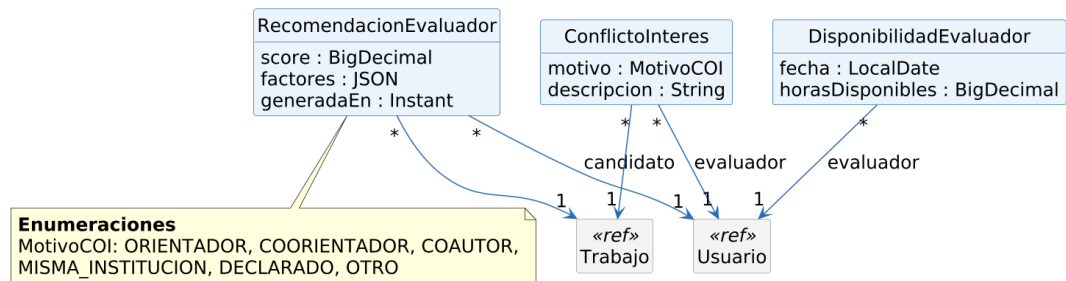


Figura 4. Diagrama de clases — Recomendación y disponibilidad, Fuente: elaboración propia

Los Diagramas de Casos de Uso describen las interacciones entre los actores del sistema —estudiantes, profesores, evaluadores externos y administradores— y las funcionalidades que cada uno utiliza. Se presentan en dos vistas según el área funcional.

La Figura 5 corresponde a la gestión de trabajos y orientación: el estudiante crea y versiona sus trabajos, consulta las sugerencias de orientadores y evaluadores y solicita la vinculación de un orientador, mientras que el profesor gestiona esas solicitudes y las plantillas de evaluación.

La Figura 6 corresponde a la evaluación y administración: los evaluadores —profesores y externos— consultan sus asignaciones y evalúan los trabajos, y el administrador gestiona usuarios, trabajos, solicitudes e importaciones.

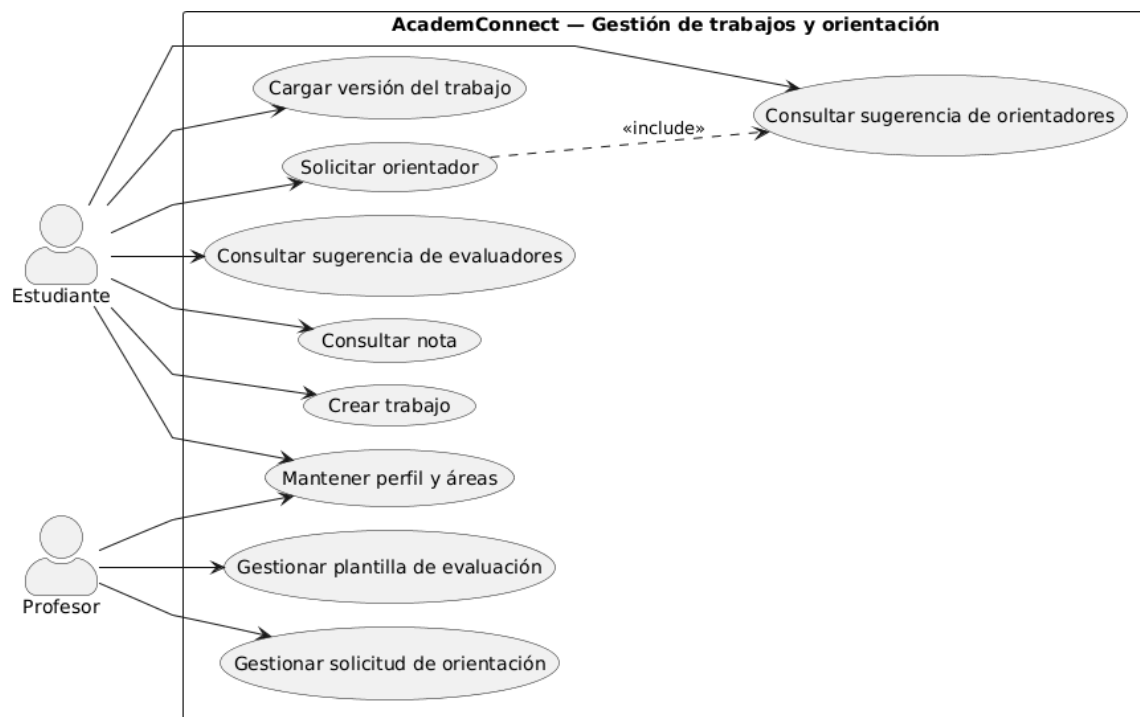


Figura 5. Diagrama de casos de uso — Gestión de trabajos y orientación, Fuente: elaboración propia

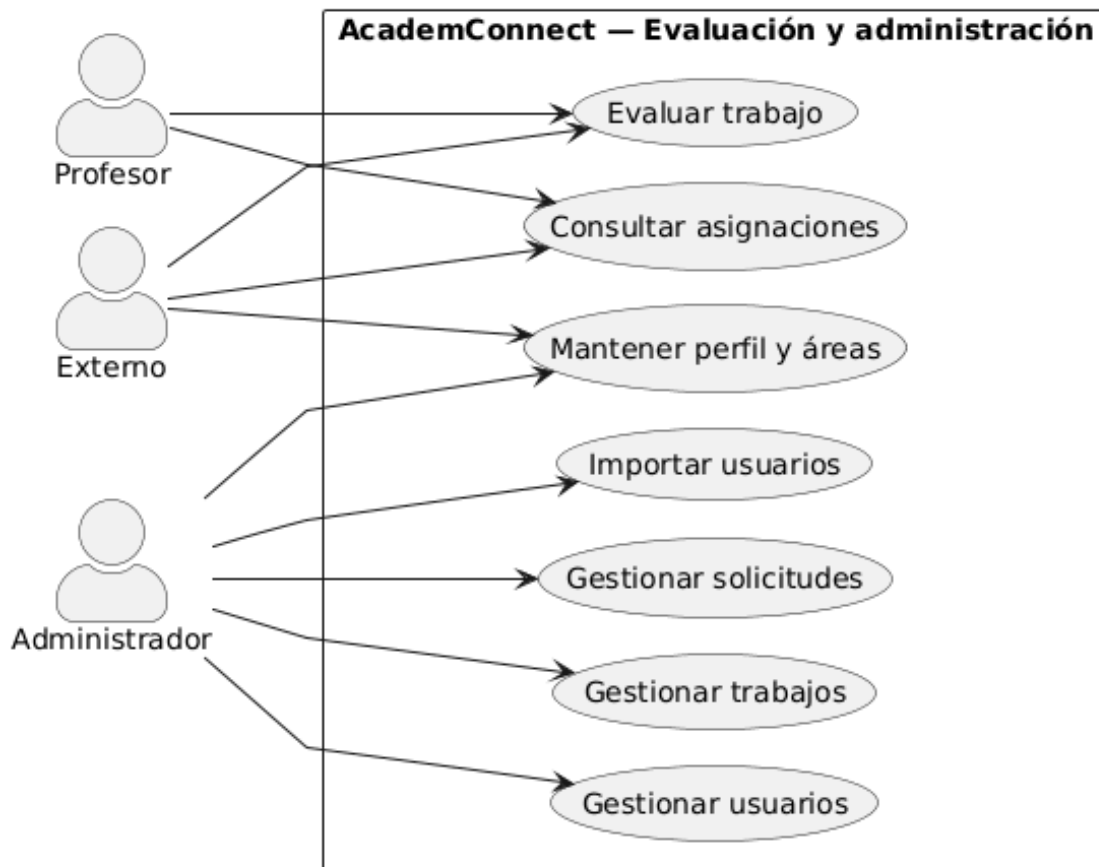


Figura 6. Diagrama de casos de uso — Evaluación y administración, Fuente: elaboración propia

Estos diagramas proporcionan una comprensión estructurada del funcionamiento del sistema previo a su implementación, facilitando la comunicación entre los diferentes miembros del equipo de desarrollo y los stakeholders del proyecto. El modelado realizado establece las bases para el diseño detallado de la base de datos, la arquitectura de la aplicación y la implementación de los módulos funcionales.

8.3. Implementación de los módulos

El sistema construido se organiza en módulos que responden a los objetivos específicos y al aporte teórico del trabajo. El repositorio institucional (objetivo 1) permite a los estudiantes registrar sus trabajos, versionarlos y almacenar los documentos asociados; cada versión conserva su archivo y sus metadatos, de modo que queda trazado el historial completo del trabajo, desde el borrador hasta la versión evaluada. El módulo de evaluaciones (objetivo 2) gestiona plantillas con criterios configurables según el tipo de trabajo y conserva una copia inmutable de la plantilla aplicada a cada evaluación, lo que garantiza que la calificación responda a los criterios vigentes en el momento de evaluar; cada criterio se califica de forma independiente y el sistema agrega los puntajes según el umbral de aprobación definido. Los perfiles académicos asocian a estudiantes, docentes y evaluadores con sus áreas de especialización y roles diferenciados; estas áreas alimentan el cálculo de afinidad temática del recomendador. El módulo de recomendación, que materializa el aporte teórico, expone a través de la *API* el algoritmo de puntuación descrito en la Formalización, sugiriendo los k mejores candidatos a revisor u orientador y registrando, para cada sugerencia, los factores que la sustentan. Por su parte, la generación de métricas del proceso evaluativo (objetivo 3) queda planificada como próxima funcionalidad, junto con la consolidación del flujo de asignación de evaluadores.

El funcionamiento de los módulos principales se ilustra a continuación mediante capturas del sistema en uso.

La Figura 7 muestra la pantalla de inicio de sesión, punto de acceso que autentica al usuario y habilita las funciones correspondientes a su rol.

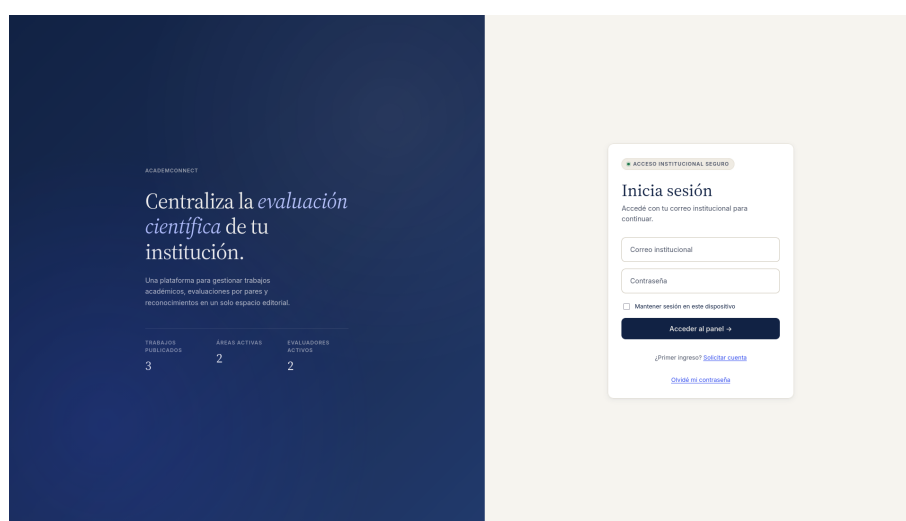


Figura 7. Pantalla de inicio de sesión, Fuente: elaboración propia

La Figura 8 presenta el repositorio institucional de trabajos publicados, con

búsqueda por texto y filtros por tipo de trabajo, año y área temática.

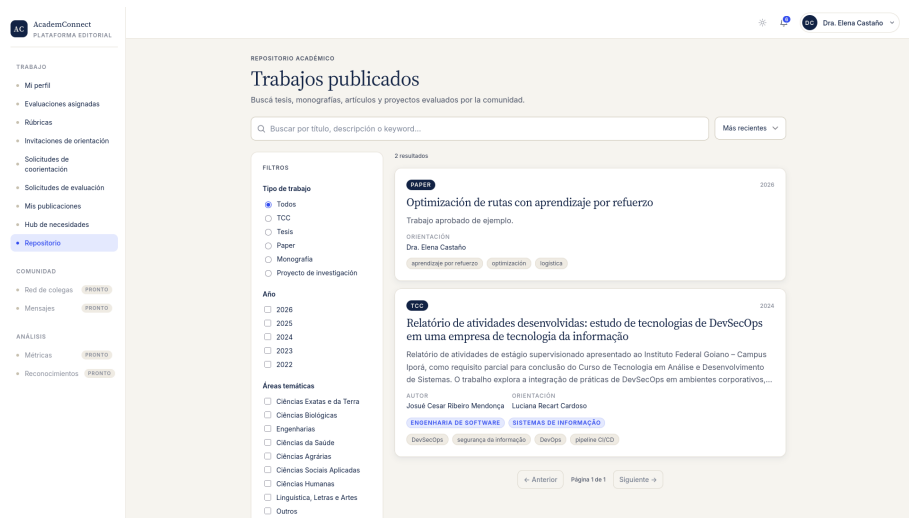


Figura 8. Repositorio institucional de trabajos publicados, Fuente: elaboración propia

La Figura 9 corresponde a la vista de un trabajo del estudiante, que reúne su descripción, el versionamiento del documento y la invitación de orientador; en esta última, la sección “Recomendados para vos” lista los docentes sugeridos por el algoritmo según su afinidad y carga.

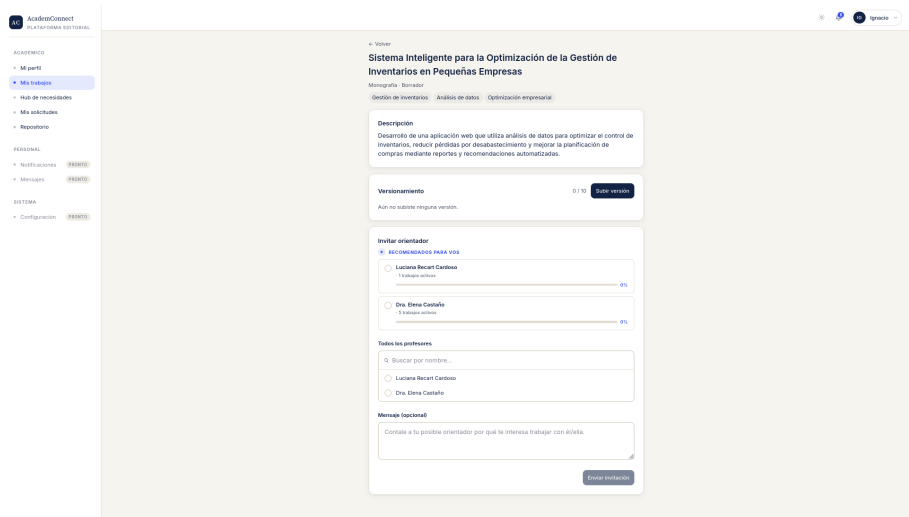


Figura 9. Gestión de un trabajo: versionamiento e invitación de orientador con recomendación, Fuente: elaboración propia

La Figura 10 corresponde al perfil académico del usuario, que reúne sus áreas de especialización, biografía y aportes; las áreas registradas aquí alimentan el cálculo de afinidad temática del recomendador.

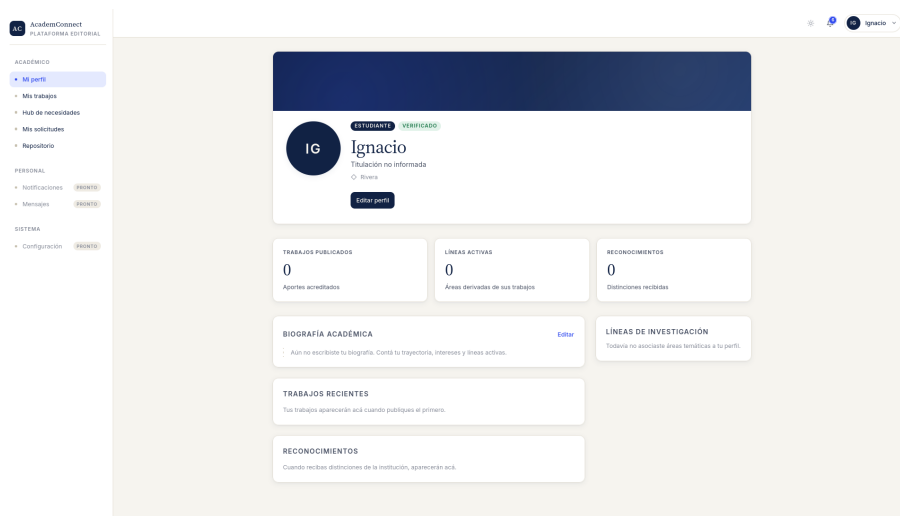


Figura 10. Perfil académico del usuario, Fuente: elaboración propia

La Figura 11 muestra la definición de una rúbrica de evaluación, donde el docente establece los criterios y sus pesos, la escala y el umbral de aprobación.

Figura 11. Definición de una rúbrica de evaluación, Fuente: elaboración propia

La Figura 12 presenta la evaluación de un trabajo: el evaluador visualiza el documento, califica cada criterio de la rúbrica y el sistema calcula la puntuación y la compara con el umbral de aprobación.

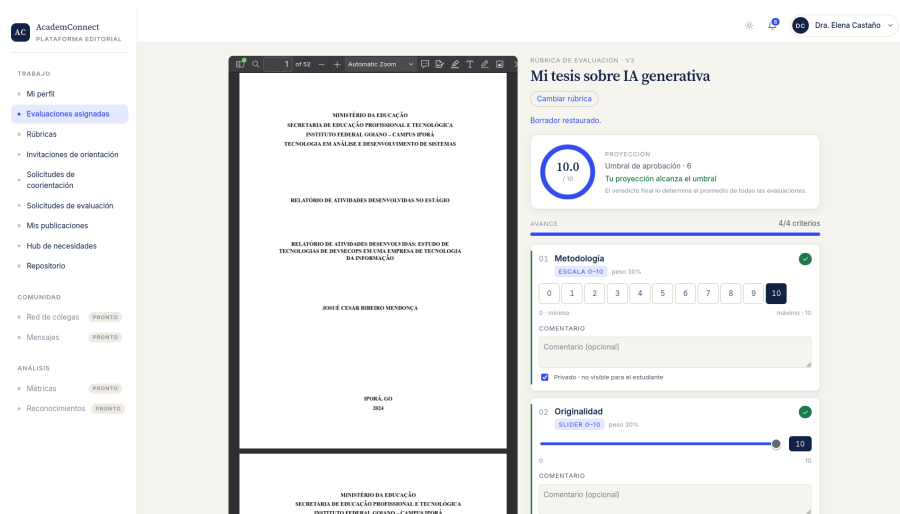


Figura 12. Evaluación de un trabajo mediante rúbrica, Fuente: elaboración propia

9. Conclusión

El presente trabajo tuvo como objetivo general desarrollar un sistema integral de gestión académica que optimizara y automatizara la evaluación de trabajos científicos. *AcademConnect* cumplió ese objetivo al integrar, en una única plataforma, un repositorio institucional, un módulo de evaluaciones dinámicas, perfiles académicos completos y un algoritmo automatizado de recomendación de revisores.

En cuanto a los objetivos específicos, se construyó el repositorio académico centralizado con versionamiento de trabajos y se implementó el módulo de evaluaciones con criterios configurables según el tipo de trabajo. Como parte del sistema se desarrollaron, además, perfiles académicos con áreas de especialización para estudiantes y docentes, que alimentan el modelo de recomendación. La generación de métricas del proceso evaluativo quedó planteada como trabajo futuro, junto con la consolidación del flujo de asignación de evaluadores.

El principal aporte del trabajo reside en el modelo configurable, transparente y trazable de recomendación de revisores, que adapta el *Reviewer Assignment Problem* al contexto de la evaluación de trabajos institucionales. De este modo, el sistema contribuye a un proceso evaluativo más ágil, equitativo y transparente, reduciendo la carga administrativa de los docentes y mejorando la trazabilidad de la producción académica de la institución.

Referencias

- [1] AKSOY, M.; YANIK, S.; AMASYALI, M. F. Reviewer Assignment Problem: A Systematic Review of the Literature. *Journal of Artificial Intelligence Research*, v. 76, p. 761-827, 2023. Disponible en: <https://doi.org/10.1613/jair.1.14318>. Consultado el: 24 de jun. de 2026.
- [2] BARRUECO, J. M.; TERMENS, M. Digital preservation in institutional repositories: a systematic literature review. *Digital Library Perspectives*, v. 38, n. 4, 2022. Disponible en: <https://doi.org/10.1108/DLP-02-2021-0011>. Consultado el: 24 de jun. de 2026.

- [3] CROW, R. The Case for Institutional Repositories: A SPARC Position Paper. Washington, DC: SPARC, 2002. Disponible en: https://sparcopen.org/wp-content/uploads/2016/01/ir_final_release.102.pdf. Consultado el: 12 de nov. de 2025.
- [4] LYNCH, C. A. Institutional Repositories: Essential Infrastructure for Scholarship in the Digital Age. *ARL: A Bimonthly Report*, n. 226, p. 1-7, 2003. Disponible en: <https://www.arl.org/resources/institutional-repositories-essential-infrastructure-for-scholarship-in-the-digital-age/>. Consultado el: 12 de nov. de 2025.
- [5] SCHULZ, R. et al. Is the future of peer review automated?. *BMC Research Notes*, v. 15, n. 203, 2022. Disponible en: <https://doi.org/10.1186/s13104-022-06080-6>. Consultado el: 24 de jun. de 2026.
- [6] STELMAKH, I.; SHAH, N. B.; SINGH, A. PeerReview4All: Fair and accurate reviewer assignment in peer review. arXiv, 2018. Disponible en: <https://arxiv.org/abs/1806.06237>. Consultado el: 24 de jun. de 2026.
- [7] KREUTZ, C. K.; SCHENKEL, R. RevASIDE: Assignment of suitable reviewer sets for publications from fixed candidate pools. arXiv, 2021. Disponible en: <https://arxiv.org/abs/2110.02862>. Consultado el: 16 de nov. de 2025.
- [8] GOSLING, J.; JOY, B.; STEELE, G.; BRACHA, G.; BUCKLEY, A.; SMITH, D. The Java Language Specification: Java SE 25 Edition. Redwood Shores: Oracle, 2025. Disponible en: <https://docs.oracle.com/javase/specs/jls/se25/html/>. Consultado el: 24 de jun. de 2026.
- [9] LINDHOLM, T.; YELLIN, F.; BRACHA, G.; BUCKLEY, A.; SMITH, D. The Java Virtual Machine Specification: Java SE 25 Edition. Redwood Shores: Oracle, 2025. Disponible en: <https://docs.oracle.com/javase/specs/jvms/se25/html/>. Consultado el: 24 de jun. de 2026.
- [10] OBE, R. O.; HSU, L. S. PostgreSQL: Up and Running. 3. ed. Sebastopol: O'Reilly Media, 2017. Disponible en: <https://www.oreilly.com/library/view/postgresql-up-and/9781491963401/>. Consultado el: 24 de jun. de 2026.
- [11] POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 Documentation. 2023. Disponible en: <https://www.postgresql.org/docs/16/>. Consultado el: 24 de jun. de 2026.
- [12] WILLINSKY, J. Open Journal Systems: An example of open source software for journal management and publishing. *Library Hi Tech*, v. 23, n. 4, p. 504-519, 2005. Disponible en: <https://doi.org/10.1108/07378830510636300>. Consultado el: 22 de nov. de 2025.
- [13] KALMUKOV, Y. Architecture of a Conference Management System Providing Advanced Paper Assignment Features. arXiv, 2011. Disponible en: <https://arxiv.org/abs/1111.6934>. Consultado el: 20 de nov. de 2025.
- [14] COLE, J.; FOSTER, H. Using Moodle: Teaching with the Popular Open Source Course Management System. 2. ed. Sebastopol: O'Reilly Media, 2007. Disponible en: <https://www.oreilly.com/library/view/using-moodle/9780596529185/>. Consultado el: 22 de nov. de 2025.

- [15] WALLS, C. Spring Boot in Action. Shelter Island: Manning Publications, 2015. Disponible en: <https://www.manning.com/books/spring-boot-in-action>. Consultado el: 24 de jun. de 2026.
- [16] GUTIERREZ, F. Pro Spring Boot 2: An Authoritative Guide to Building Microservices, Web and Enterprise Applications, and Best Practices. 2. ed. New York: Apress, 2019. Disponible en: <https://link.springer.com/book/10.1007/978-1-4842-3676-4>. Consultado el: 21 de nov. de 2025.
- [17] FAIN, Y.; MOISEEV, A. Angular Development with TypeScript. 2. ed. Shelter Island: Manning Publications, 2018. Disponible en: <https://www.manning.com/books/angular-development-with-typescript-second-edition>. Consultado el: 24 de jun. de 2026.
- [18] FOWLER, M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley Professional, 2002. Disponible en: <https://martinfowler.com/books/ea.html>. Consultado el: 24 de jun. de 2026.
- [19] BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. The Unified Modeling Language User Guide. 2. ed. Boston: Addison-Wesley Professional, 2005. Disponible en: <https://www.uml.org/>. Consultado el: 12 de nov. de 2025.
- [20] JACCARD, P. The distribution of the flora in the alpine zone. *New Phytologist*, v. 11, n. 2, p. 37-50, 1912. Disponible en: <https://doi.org/10.1111/j.1469-8137.1912.tb05611.x>. Consultado el: 24 de jun. de 2026.
- [21] JONES, M.; BRADLEY, J.; SAKIMURA, N. JSON Web Token (JWT). RFC 7519. [S.l.]: IETF, 2015. Disponible en: <https://www.rfc-editor.org/rfc/rfc7519>. Consultado el: 24 de jun. de 2026.
- [22] HARDT, D. (Ed.). The OAuth 2.0 Authorization Framework. RFC 6749. [S.l.]: IETF, 2012. Disponible en: <https://www.rfc-editor.org/rfc/rfc6749>. Consultado el: 24 de jun. de 2026.
- [23] NOTTINGHAM, M.; WILDE, E.; DALAL, S. Problem Details for HTTP APIs. RFC 9457. [S.l.]: IETF, 2023. Disponible en: <https://www.rfc-editor.org/rfc/rfc9457>. Consultado el: 24 de jun. de 2026.
- [24] MERKEL, D. Docker: lightweight Linux containers for consistent development and deployment. *Linux Journal*, v. 2014, n. 239, 2014. Disponible en: <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>. Consultado el: 24 de jun. de 2026.